

Note 5.41 is due to Vazirani and Vazirani. Alexi et al. [23] proved analogous results for the *modified-Rabin generator*, which differs as follows from the Blum-Blum-Shub generator: in step 3.1 of Algorithm 5.40, let $\bar{x} = x_{i-1}^2 \bmod n$; if $\bar{x} < n/2$, then $x_i = \bar{x}$; otherwise, $x_i = n - \bar{x}$.

Impagliazzo and Naor [569] devised efficient constructions for a CSPRBG and for a universal one-way hash function which are provably as secure as the subset sum problem. Fischer and Stern [411] presented a simple and efficient CSPRBG which is provably as secure as the *syndrome decoding problem*.

Yao [1258] showed how to obtain a CSPRBG using any one-way permutation. Levin [761] generalized this result and showed how to obtain a CSPRBG using any one-way function. For further refinements, see Goldreich, Krawczyk, and Luby [470], Impagliazzo, Levin, and Luby [568], and Håstad [545].

A *random function* $f: \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a function which assigns independent and random values $f(x) \in \{0, 1\}^n$ to all arguments $x \in \{0, 1\}^n$. Goldreich, Goldwasser, and Micali [468] introduced a computational complexity measure of the randomness of functions. They defined a function to be *poly-random* if no polynomial-time algorithm can distinguish between values of the function and true random strings, even when the algorithm is permitted to select the arguments to the function. Goldreich, Goldwasser, and Micali presented an algorithm for constructing poly-random functions assuming the existence of one-way functions. This theory was applied by Goldreich, Goldwasser, and Micali [467] to develop provably secure protocols for the (essentially) storageless distribution of secret identification numbers, message authentication with timestamping, dynamic hashing, and identify friend or foe systems. Luby and Rackoff [776] showed how poly-random permutations can be efficiently constructed from poly-random functions. This result was used, together with some of the design principles of DES, to show how any CSPRBG can be used to construct a symmetric-key block cipher which is provably secure against chosen-plaintext attack. A simplified and generalized treatment of Luby and Rackoff's construction was given by Maurer [816].

Schnorr [1096] used Luby and Rackoff's poly-random permutation generator to construct a pseudorandom bit generator that was claimed to pass all statistical tests depending only on a small fraction of the output sequence, even when infinite computational resources are available. Rueppel [1079] showed that this claim is erroneous, and demonstrated that the generator can be distinguished from a truly random bit generator using only a small number of output bits. Maurer and Massey [821] extended Schnorr's work, and proved the existence of pseudorandom bit generators that pass all statistical tests depending only on a small fraction of the output sequence, even when infinite computational resources are available. The security of the generators does not rely on any unproved hypothesis, but rather on the assumption that the adversary can access only a limited number of bits of the generated sequence. This work is primarily of theoretical interest since no such polynomial-time generators are known.

Chapter

6

Stream Ciphers

Contents in Brief

6.1	Introduction	191
6.2	Feedback shift registers	195
6.3	Stream ciphers based on LFSRs	203
6.4	Other stream ciphers	212
6.5	Notes and further references	216

6.1 Introduction

Stream ciphers are an important class of encryption algorithms. They encrypt individual characters (usually binary digits) of a plaintext message one at a time, using an encryption transformation which varies with time. By contrast, *block ciphers* (Chapter 7) tend to simultaneously encrypt groups of characters of a plaintext message using a fixed encryption transformation. Stream ciphers are generally faster than block ciphers in hardware, and have less complex hardware circuitry. They are also more appropriate, and in some cases mandatory (e.g., in some telecommunications applications), when buffering is limited or when characters must be individually processed as they are received. Because they have limited or no error propagation, stream ciphers may also be advantageous in situations where transmission errors are highly probable.

There is a vast body of theoretical knowledge on stream ciphers, and various design principles for stream ciphers have been proposed and extensively analyzed. However, there are relatively few fully-specified stream cipher algorithms in the open literature. This unfortunate state of affairs can partially be explained by the fact that most stream ciphers used in practice tend to be proprietary and confidential. By contrast, numerous concrete block cipher proposals have been published, some of which have been standardized or placed in the public domain. Nevertheless, because of their significant advantages, stream ciphers are widely used today, and one can expect increasingly more concrete proposals in the coming years.

Chapter outline

The remainder of §6.1 introduces basic concepts relevant to stream ciphers. Feedback shift registers, in particular linear feedback shift registers (LFSRs), are the basic building block in most stream ciphers that have been proposed; they are studied in §6.2. Three general techniques for utilizing LFSRs in the construction of stream ciphers are presented in §6.3: using

a nonlinear combining function on the outputs of several LFSRs (§6.3.1), using a nonlinear filtering function on the contents of a single LFSR (§6.3.2), and using the output of one (or more) LFSRs to control the clock of one (or more) other LFSRs (§6.3.3). Two concrete proposals for clock-controlled generators, the alternating step generator and the shrinking generator are presented in §6.3.3. §6.4 presents a stream cipher not based on LFSRs, namely SEAL. §6.5 concludes with references and further chapter notes.

6.1.1 Classification

Stream ciphers can be either symmetric-key or public-key. The focus of this chapter is symmetric-key stream ciphers; the Blum-Goldwasser probabilistic public-key encryption scheme (§8.7.2) is an example of a public-key stream cipher.

6.1 Note (block vs. stream ciphers) Block ciphers process plaintext in relatively large blocks (e.g., $n \geq 64$ bits). The same function is used to encrypt successive blocks; thus (pure) block ciphers are *memoryless*. In contrast, stream ciphers process plaintext in blocks as small as a single bit, and the encryption function may vary as plaintext is processed; thus stream ciphers are said to have memory. They are sometimes called *state ciphers* since encryption depends on not only the key and plaintext, but also on the current state. This distinction between block and stream ciphers is not definitive (see Remark 7.25); adding a small amount of memory to a block cipher (as in the CBC mode) results in a stream cipher with large blocks.

(i) The one-time pad

Recall (Definition 1.39) that a *Vernam cipher* over the binary alphabet is defined by

$$c_i = m_i \oplus k_i \text{ for } i = 1, 2, 3, \dots,$$

where $m_1, m_2, m_3, \dots$ are the plaintext digits, $k_1, k_2, k_3, \dots$ (the *keystream*) are the key digits, $c_1, c_2, c_3, \dots$ are the ciphertext digits, and $\oplus$ is the XOR function (bitwise addition modulo 2). Decryption is defined by $m_i = c_i \oplus k_i$. If the keystream digits are generated independently and randomly, the Vernam cipher is called a *one-time pad*, and is unconditionally secure (§1.13.3(i)) against a ciphertext-only attack. More precisely, if M , C , and K are random variables respectively denoting the plaintext, ciphertext, and secret key, and if $H(\cdot)$ denotes the entropy function (Definition 2.39), then $H(M|C) = H(M)$. Equivalently, $I(M; C) = 0$ (see Definition 2.45): the ciphertext contributes no information about the plaintext.

Shannon proved that a necessary condition for a symmetric-key encryption scheme to be unconditionally secure is that $H(K) \geq H(M)$. That is, the uncertainty of the secret key must be at least as great as the uncertainty of the plaintext. If the key has bitlength k , and the key bits are chosen randomly and independently, then $H(K) = k$, and Shannon's necessary condition for unconditional security becomes $k \geq H(M)$. The one-time pad is unconditionally secure regardless of the statistical distribution of the plaintext, and is optimal in the sense that its key is the smallest possible among all symmetric-key encryption schemes having this property.

An obvious drawback of the one-time pad is that the key should be as long as the plaintext, which increases the difficulty of key distribution and key management. This motivates the design of stream ciphers where the keystream is *pseudorandomly* generated from a smaller secret key, with the intent that the keystream appears random to a computationally bounded adversary. Such stream ciphers do not offer unconditional security (since $H(K) \ll H(M)$), but the hope is that they are computationally secure (§1.13.3(iv)).

Stream ciphers are commonly classified as being *synchronous* or *self-synchronizing*.

(ii) Synchronous stream ciphers

6.2 Definition A *synchronous* stream cipher is one in which the keystream is generated independently of the plaintext message and of the ciphertext.

The encryption process of a synchronous stream cipher can be described by the equations

$$\begin{aligned}\sigma_{i+1} &= f(\sigma_i, k), \\ z_i &= g(\sigma_i, k), \\ c_i &= h(z_i, m_i),\end{aligned}$$

where σ_0 is the *initial state* and may be determined from the key k , f is the *next-state function*, g is the function which produces the *keystream* z_i , and h is the *output function* which combines the keystream and plaintext m_i to produce ciphertext c_i . The encryption and decryption processes are depicted in Figure 6.1. The OFB mode of a block cipher (see §7.2.2(iv)) is an example of a synchronous stream cipher.

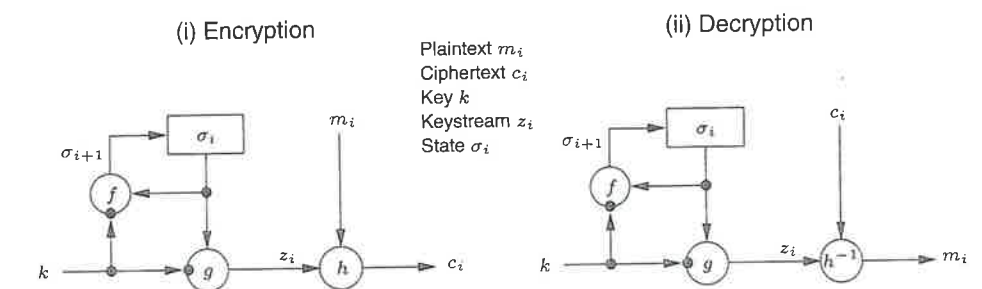


Figure 6.1: General model of a synchronous stream cipher.

6.3 Note (properties of synchronous stream ciphers)

- (i) *synchronization requirements*. In a synchronous stream cipher, both the sender and receiver must be *synchronized* – using the same key and operating at the same position (state) within that key – to allow for proper decryption. If synchronization is lost due to ciphertext digits being inserted or deleted during transmission, then decryption fails and can only be restored through additional techniques for re-synchronization. Techniques for re-synchronization include re-initialization, placing special markers at regular intervals in the ciphertext, or, if the plaintext contains enough redundancy, trying all possible keystream offsets.
- (ii) *no error propagation*. A ciphertext digit that is modified (but not deleted) during transmission does not affect the decryption of other ciphertext digits.
- (iii) *active attacks*. As a consequence of property (i), the insertion, deletion, or replay of ciphertext digits by an active adversary causes immediate loss of synchronization, and hence might possibly be detected by the decryptor. As a consequence of property (ii), an active adversary might possibly be able to make changes to selected ciphertext digits, and know exactly what affect these changes have on the plaintext. This illustrates that additional mechanisms must be employed in order to provide data origin authentication and data integrity guarantees (see §9.5.4).

Most of the stream ciphers that have been proposed to date in the literature are additive stream ciphers, which are defined below.

6.4 Definition A *binary additive stream cipher* is a synchronous stream cipher in which the keystream, plaintext, and ciphertext digits are binary digits, and the output function h is the XOR function.

Binary additive stream ciphers are depicted in Figure 6.2. Referring to Figure 6.2, the *keystream generator* is composed of the next-state function f and the function g (see Figure 6.1), and is also known as the *running key generator*.

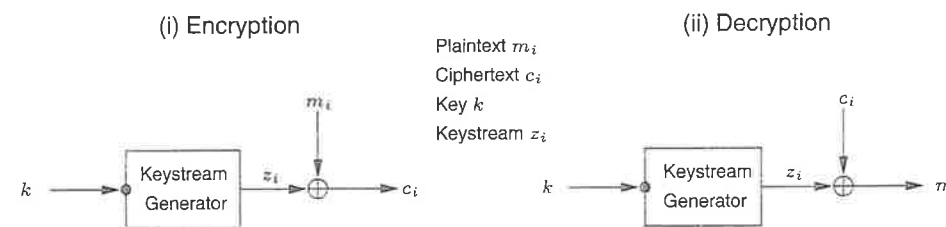


Figure 6.2: General model of a binary additive stream cipher.

(iii) Self-synchronizing stream ciphers

6.5 Definition A *self-synchronizing* or *asynchronous* stream cipher is one in which the keystream is generated as a function of the key and a fixed number of previous ciphertext digits.

The encryption function of a self-synchronizing stream cipher can be described by the equations

$$\begin{aligned}\sigma_i &= (c_{i-t}, c_{i-t+1}, \dots, c_{i-1}), \\ z_i &= g(\sigma_i, k), \\ c_i &= h(z_i, m_i),\end{aligned}$$

where $\sigma_0 = (c_{-t}, c_{-t+1}, \dots, c_{-1})$ is the (non-secret) *initial state*, k is the *key*, g is the function which produces the *keystream* z_i , and h is the *output function* which combines the keystream and plaintext m_i to produce ciphertext c_i . The encryption and decryption processes are depicted in Figure 6.3. The most common presently-used self-synchronizing stream ciphers are based on block ciphers in 1-bit cipher feedback mode (see §7.2.2(iii)).

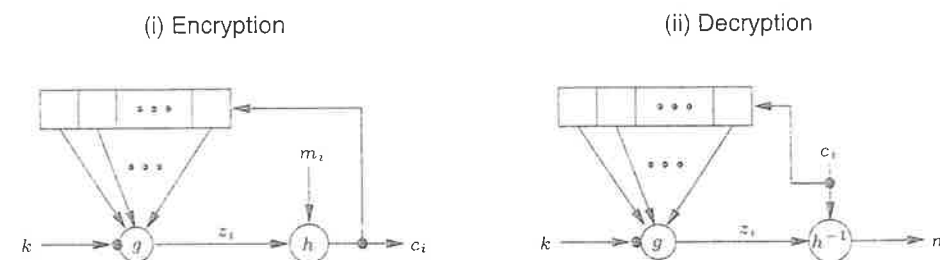


Figure 6.3: General model of a self-synchronizing stream cipher.

6.6 Note (properties of self-synchronizing stream ciphers)

- (i) *self-synchronization*. Self-synchronization is possible if ciphertext digits are deleted or inserted, because the decryption mapping depends only on a fixed number of preceding ciphertext characters. Such ciphers are capable of re-establishing proper decryption automatically after loss of synchronization, with only a fixed number of plaintext characters unrecoverable.
- (ii) *limited error propagation*. Suppose that the state of a self-synchronization stream cipher depends on t previous ciphertext digits. If a single ciphertext digit is modified (or even deleted or inserted) during transmission, then decryption of up to t subsequent ciphertext digits may be incorrect, after which correct decryption resumes.
- (iii) *active attacks*. Property (ii) implies that any modification of ciphertext digits by an active adversary causes several other ciphertext digits to be decrypted incorrectly, thereby improving (compared to synchronous stream ciphers) the likelihood of being detected by the decryptor. As a consequence of property (i), it is more difficult (than for synchronous stream ciphers) to detect insertion, deletion, or replay of ciphertext digits by an active adversary. This illustrates that additional mechanisms must be employed in order to provide data origin authentication and data integrity guarantees (see §9.5.4).
- (iv) *diffusion of plaintext statistics*. Since each plaintext digit influences the entire following ciphertext, the statistical properties of the plaintext are dispersed through the ciphertext. Hence, self-synchronizing stream ciphers may be more resistant than synchronous stream ciphers against attacks based on plaintext redundancy.

6.2 Feedback shift registers

Feedback shift registers, in particular linear feedback shift registers, are the basic components of many keystream generators. §6.2.1 introduces linear feedback shift registers. The linear complexity of binary sequences is studied in §6.2.2, while the Berlekamp-Massey algorithm for computing it is presented in §6.2.3. Finally, nonlinear feedback shift registers are discussed in §6.2.4.

6.2.1 Linear feedback shift registers

Linear feedback shift registers (LFSRs) are used in many of the keystream generators that have been proposed in the literature. There are several reasons for this:

1. LFSRs are well-suited to hardware implementation;
2. they can produce sequences of large period (Fact 6.12);
3. they can produce sequences with good statistical properties (Fact 6.14); and
4. because of their structure, they can be readily analyzed using algebraic techniques.

6.7 Definition A *linear feedback shift register* (LFSR) of length L consists of L *stages* (or *delay elements*) numbered $0, 1, \dots, L-1$, each capable of storing one bit and having one input and one output; and a clock which controls the movement of data. During each unit of time the following operations are performed:

- (i) the content of stage 0 is output and forms part of the *output sequence*;

- (ii) the content of stage i is moved to stage $i - 1$ for each $i, 1 \leq i \leq L - 1$; and
- (iii) the new content of stage $L - 1$ is the *feedback bit* s_j which is calculated by adding together modulo 2 the previous contents of a fixed subset of stages $0, 1, \dots, L - 1$.

Figure 6.4 depicts an LFSR. Referring to the figure, each c_i is either 0 or 1; the closed semi-circles are AND gates; and the feedback bit s_j is the modulo 2 sum of the contents of those stages $i, 0 \leq i \leq L - 1$, for which $c_{L-i} = 1$.

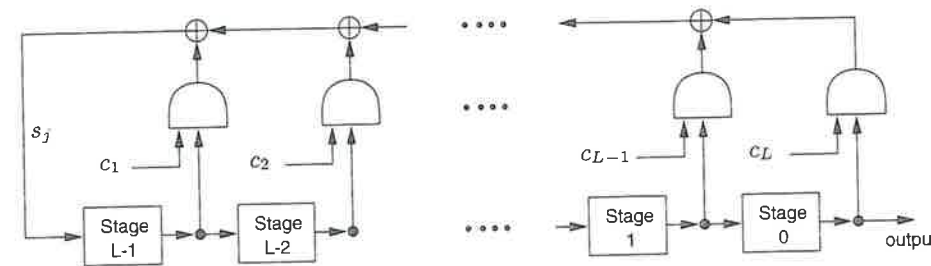


Figure 6.4: A linear feedback shift register (LFSR) of length L .

6.8 Definition The LFSR of Figure 6.4 is denoted $\langle L, C(D) \rangle$, where $C(D) = 1 + c_1D + c_2D^2 + \dots + c_LD^L \in \mathbb{Z}_2[D]$ is the *connection polynomial*. The LFSR is said to be *non-singular* if the degree of $C(D)$ is L (that is, $c_L = 1$). If the initial content of stage i is $s_i \in \{0, 1\}$ for each $i, 0 \leq i \leq L - 1$, then $[s_{L-1}, \dots, s_1, s_0]$ is called the *initial state* of the LFSR.

6.9 Fact If the initial state of the LFSR in Figure 6.4 is $[s_{L-1}, \dots, s_1, s_0]$, then the output sequence $s = s_0, s_1, s_2, \dots$ is uniquely determined by the following recursion:

$$s_j = (c_1s_{j-1} + c_2s_{j-2} + \dots + c_Ls_{j-L}) \bmod 2 \text{ for } j \geq L.$$

6.10 Example (output sequence of an LFSR) Consider the LFSR $\langle 4, 1 + D + D^4 \rangle$ depicted in Figure 6.5. If the initial state of the LFSR is $[0, 0, 0, 0]$, the output sequence is the zero sequence. The following tables show the contents of the stages D_3, D_2, D_1, D_0 at the end of each unit of time t when the initial state is $[0, 1, 1, 0]$.

t	D_3	D_2	D_1	D_0
0	0	1	1	0
1	0	0	1	1
2	1	0	0	1
3	0	1	0	0
4	0	0	1	0
5	0	0	0	1
6	1	0	0	0
7	1	1	0	0

t	D_3	D_2	D_1	D_0
8	1	1	1	0
9	1	1	1	1
10	0	1	1	1
11	1	0	1	1
12	0	1	0	1
13	1	0	1	0
14	1	1	0	1
15	0	1	1	0

The output sequence is $s = 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, \dots$, and is periodic with period 15 (see Definition 5.25). $\square$

The significance of an LFSR being non-singular is explained by Fact 6.11.

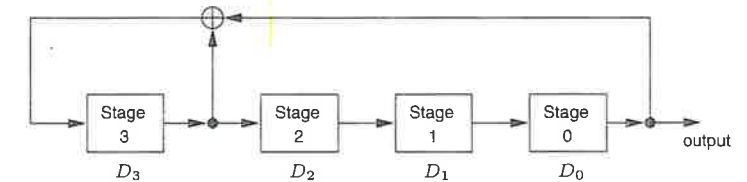


Figure 6.5: The LFSR $\langle 4, 1 + D + D^4 \rangle$ of Example 6.10.

6.11 Fact Every output sequence (i.e., for all possible initial states) of an LFSR $\langle L, C(D) \rangle$ is periodic if and only if the connection polynomial $C(D)$ has degree L .

If an LFSR $\langle L, C(D) \rangle$ is *singular* (i.e., $C(D)$ has degree less than L), then not all output sequences are periodic. However, the output sequences are *ultimately periodic*; that is, the sequences obtained by ignoring a certain finite number of terms at the beginning are periodic. For the remainder of this chapter, it will be assumed that all LFSRs are non-singular. Fact 6.12 determines the periods of the output sequences of some special types of non-singular LFSRs.

6.12 Fact (periods of LFSR output sequences) Let $C(D) \in \mathbb{Z}_2[D]$ be a connection polynomial of degree L .

- (i) If $C(D)$ is irreducible over $\mathbb{Z}_2$ (see Definition 2.190), then each of the $2^L - 1$ non-zero initial states of the non-singular LFSR $\langle L, C(D) \rangle$ produces an output sequence with period equal to the least positive integer N such that $C(D)$ divides $1 + D^N$ in $\mathbb{Z}_2[D]$. (Note: it is always the case that this N is a divisor of $2^L - 1$.)
- (ii) If $C(D)$ is a primitive polynomial (see Definition 2.228), then each of the $2^L - 1$ non-zero initial states of the non-singular LFSR $\langle L, C(D) \rangle$ produces an output sequence with maximum possible period $2^L - 1$.

A method for generating primitive polynomials over $\mathbb{Z}_2$ uniformly at random is given in Algorithm 4.78. Table 4.8 lists a primitive polynomial of degree m over $\mathbb{Z}_2$ for each $m, 1 \leq m \leq 229$. Fact 6.12(ii) motivates the following definition.

6.13 Definition If $C(D) \in \mathbb{Z}_2[D]$ is a primitive polynomial of degree L , then $\langle L, C(D) \rangle$ is called a *maximum-length LFSR*. The output of a maximum-length LFSR with non-zero initial state is called an *m-sequence*.

Fact 6.14 demonstrates that the output sequences of maximum-length LFSRs have good statistical properties.

6.14 Fact (statistical properties of m-sequences) Let s be an *m-sequence* that is generated by a maximum-length LFSR of length L .

- (i) Let k be an integer, $1 \leq k \leq L$, and let $\bar{s}$ be any subsequence of s of length $2^L + k - 2$. Then each non-zero sequence of length k appears exactly 2^{L-k} times as a subsequence of $\bar{s}$. Furthermore, the zero sequence of length k appears exactly $2^{L-k} - 1$ times as a subsequence of $\bar{s}$. In other words, the distribution of patterns having fixed length of at most L is almost uniform.
- (ii) s satisfies Golomb's randomness postulates (§5.4.3). That is, every *m-sequence* is also a *pn-sequence* (see Definition 5.29).

6.15 Example (*m-sequence*) Since $C(D) = 1 + D + D^4$ is a primitive polynomial over $\mathbb{Z}_2$, the LFSR $\langle 4, 1 + D + D^4 \rangle$ is a maximum-length LFSR. Hence, the output sequence of this LFSR is an *m*-sequence of maximum possible period $N = 2^4 - 1 = 15$ (cf. Example 6.10). Example 5.30 verifies that this output sequence satisfies Golomb's randomness properties. $\square$

6.2.2 Linear complexity

This subsection summarizes selected results about the linear complexity of sequences. All sequences are assumed to be binary sequences. Notation: s denotes an infinite sequence whose terms are $s_0, s_1, s_2, \dots$; s^n denotes a finite sequence of length n whose terms are $s_0, s_1, \dots, s_{n-1}$ (see Definition 5.24).

6.16 Definition An LFSR is said to *generate* a sequence s if there is some initial state for which the output sequence of the LFSR is s . Similarly, an LFSR is said to *generate* a finite sequence s^n if there is some initial state for which the output sequence of the LFSR has s^n as its first n terms.

6.17 Definition The *linear complexity* of an infinite binary sequence s , denoted $L(s)$, is defined as follows:

- (i) if s is the zero sequence $s = 0, 0, 0, \dots$, then $L(s) = 0$;
- (ii) if no LFSR generates s , then $L(s) = \infty$;
- (iii) otherwise, $L(s)$ is the length of the shortest LFSR that generates s .

6.18 Definition The *linear complexity* of a finite binary sequence s^n , denoted $L(s^n)$, is the length of the shortest LFSR that generates a sequence having s^n as its first n terms.

Facts 6.19 – 6.22 summarize some basic results about linear complexity.

6.19 Fact (*properties of linear complexity*) Let s and t be binary sequences.

- (i) For any $n \geq 1$, the linear complexity of the subsequence s^n satisfies $0 \leq L(s^n) \leq n$.
- (ii) $L(s^n) = 0$ if and only if s^n is the zero sequence of length n .
- (iii) $L(s^n) = n$ if and only if $s^n = 0, 0, 0, \dots, 0, 1$.
- (iv) If s is periodic with period N , then $L(s) \leq N$.
- (v) $L(s \oplus t) \leq L(s) + L(t)$, where $s \oplus t$ denotes the bitwise XOR of s and t .

6.20 Fact If the polynomial $C(D) \in \mathbb{Z}_2[D]$ is irreducible over $\mathbb{Z}_2$ and has degree L , then each of the $2^L - 1$ non-zero initial states of the non-singular LFSR $\langle L, C(D) \rangle$ produces an output sequence with linear complexity L .

6.21 Fact (*expectation and variance of the linear complexity of a random sequence*) Let s^n be chosen uniformly at random from the set of all binary sequences of length n , and let $L(s^n)$ be the linear complexity of s^n . Let $B(n)$ denote the parity function: $B(n) = 0$ if n is even; $B(n) = 1$ if n is odd.

- (i) The expected linear complexity of s^n is

$$E(L(s^n)) = \frac{n}{2} + \frac{4+B(n)}{18} - \frac{1}{2^n} \left(\frac{n}{3} + \frac{2}{9} \right).$$

Hence, for moderately large n , $E(L(s^n)) \approx \frac{n}{2} + \frac{2}{9}$ if n is even, and $E(L(s^n)) \approx \frac{n}{2} + \frac{5}{18}$ if n is odd.

- (ii) The variance of the linear complexity of s^n is $\text{Var}(L(s^n)) =$

$$\frac{86}{81} - \frac{1}{2^n} \left(\frac{14-B(n)}{27}n + \frac{82-2B(n)}{81} \right) - \frac{1}{2^{2n}} \left(\frac{1}{9}n^2 + \frac{4}{27}n + \frac{4}{81} \right).$$

Hence, $\text{Var}(L(s^n)) \approx \frac{86}{81}$ for moderately large n .

6.22 Fact (*expectation of the linear complexity of a random periodic sequence*) Let s^n be chosen uniformly at random from the set of all binary sequences of length n , where $n = 2^t$ for some fixed $t \geq 1$, and let s be the n -periodic infinite sequence obtained by repeating the sequence s^n . Then the expected linear complexity of s is $E(L(s^n)) = n - 1 + 2^{-n}$.

The linear complexity profile of a binary sequence is introduced next.

6.23 Definition Let $s = s_0, s_1, \dots$ be a binary sequence, and let L_N denote the linear complexity of the subsequence $s^N = s_0, s_1, \dots, s_{N-1}$, $N \geq 0$. The sequence $L_1, L_2, \dots$ is called the *linear complexity profile* of s . Similarly, if $s^n = s_0, s_1, \dots, s_{n-1}$ is a finite binary sequence, the sequence $L_1, L_2, \dots, L_n$ is called the *linear complexity profile* of s^n .

The linear complexity profile of a sequence can be computed using the Berlekamp-Massey algorithm (Algorithm 6.30); see also Note 6.31. The following properties of the linear complexity profile can be deduced from Fact 6.29.

6.24 Fact (*properties of linear complexity profile*) Let $L_1, L_2, \dots$ be the linear complexity profile of a sequence $s = s_0, s_1, \dots$.

- (i) If $j > i$, then $L_j \geq L_i$.
- (ii) $L_{N+1} > L_N$ is possible only if $L_N \leq N/2$.
- (iii) If $L_{N+1} > L_N$, then $L_{N+1} + L_N = N + 1$.

The linear complexity profile of a sequence s can be graphed by plotting the points (N, L_N) , $N \geq 1$, in the $N \times L$ plane and joining successive points by a horizontal line followed by a vertical line, if necessary (see Figure 6.6). Fact 6.24 can then be interpreted as saying that the graph of a linear complexity profile is non-decreasing. Moreover, a (vertical) jump in the graph can only occur from below the line $L = N/2$; if a jump occurs, then it is symmetric about this line. Fact 6.25 shows that the expected linear complexity of a random sequence should closely follow the line $L = N/2$.

6.25 Fact (*expected linear complexity profile of a random sequence*) Let $s = s_0, s_1, \dots$ be a random sequence, and let L_N be the linear complexity of the subsequence $s^N = s_0, s_1, \dots, s_{N-1}$ for each $N \geq 1$. For any fixed index $N \geq 1$, the expected smallest j for which $L_{N+j} > L_N$ is 2 if $L_N \leq N/2$, or $2 + 2L_N - N$ if $L_N > N/2$. Moreover, the expected increase in linear complexity is 2 if $L_N \geq N/2$, or $N - 2L_N + 2$ if $L_N < N/2$.

6.26 Example (*linear complexity profile*) Consider the 20-periodic sequence s with cycle

$$s^{20} = 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0.$$

The linear complexity profile of s is 1, 1, 1, 3, 3, 3, 5, 5, 5, 6, 6, 6, 8, 8, 8, 9, 9, 10, 10, 11, 11, 11, 14, 14, 14, 14, 15, 15, 15, 17, 17, 17, 18, 18, 19, 19, 19, 19, $\dots$. Figure 6.6 shows the graph of the linear complexity profile of s . $\square$

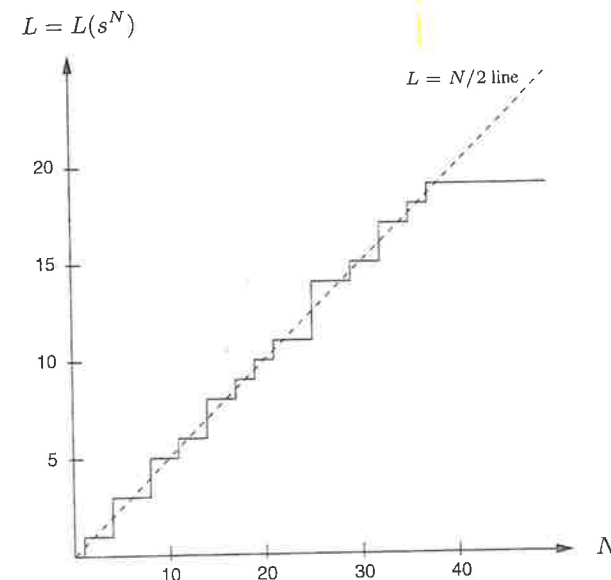


Figure 6.6: Linear complexity profile of the 20-periodic sequence of Example 6.26.

As is the case with all statistical tests for randomness (cf. §5.4), the condition that a sequence s have a linear complexity profile that closely resembles that of a random sequence is *necessary* but not *sufficient* for s to be considered random. This point is illustrated in the following example.

6.27 Example (*limitations of the linear complexity profile*) The linear complexity profile of the sequence s defined as

$$s_i = \begin{cases} 1, & \text{if } i = 2^j - 1 \text{ for some } j \geq 0, \\ 0, & \text{otherwise,} \end{cases}$$

follows the line $L = N/2$ as closely as possible. That is, $L(s^N) = \lfloor (N+1)/2 \rfloor$ for all $N \geq 1$. However, the sequence s is clearly non-random. $\square$

6.2.3 Berlekamp-Massey algorithm

The Berlekamp-Massey algorithm (Algorithm 6.30) is an efficient algorithm for determining the linear complexity of a finite binary sequence s^n of length n (see Definition 6.18). The algorithm takes n iterations, with the N th iteration computing the linear complexity of the subsequence s^N consisting of the first N terms of s^n . The theoretical basis for the algorithm is Fact 6.29.

6.28 Definition Consider the finite binary sequence $s^{N+1} = s_0, s_1, \dots, s_{N-1}, s_N$. For $C(D) = 1 + c_1D + \dots + c_LD^L$, let $\langle L, C(D) \rangle$ be an LFSR that generates the subsequence $s^N = s_0, s_1, \dots, s_{N-1}$. The *next discrepancy* d_N is the difference between s_N and the $(N+1)$ st term generated by the LFSR: $d_N = (s_N + \sum_{i=1}^L c_i s_{N-i}) \bmod 2$.

6.29 Fact Let $s^N = s_0, s_1, \dots, s_{N-1}$ be a finite binary sequence of linear complexity $L = L(s^N)$, and let $\langle L, C(D) \rangle$ be an LFSR which generates s^N .

- (i) The LFSR $\langle L, C(D) \rangle$ also generates $s^{N+1} = s_0, s_1, \dots, s_{N-1}, s_N$ if and only if the next discrepancy d_N is equal to 0.
- (ii) If $d_N = 0$, then $L(s^{N+1}) = L$.
- (iii) Suppose $d_N = 1$. Let m the largest integer $< N$ such that $L(s^m) < L(s^N)$, and let $\langle L(s^m), B(D) \rangle$ be an LFSR of length $L(s^m)$ which generates s^m . Then $\langle L', C'(D) \rangle$ is an LFSR of smallest length which generates s^{N+1} , where

$$L' = \begin{cases} L, & \text{if } L > N/2, \\ N+1-L, & \text{if } L \leq N/2, \end{cases}$$

$$\text{and } C'(D) = C(D) + B(D) \cdot D^{N-m}.$$

6.30 Algorithm Berlekamp-Massey algorithm

INPUT: a binary sequence $s^n = s_0, s_1, s_2, \dots, s_{n-1}$ of length n .

OUTPUT: the linear complexity $L(s^n)$ of s^n , $0 \leq L(s^n) \leq n$.

1. *Initialization.* $C(D) \leftarrow 1$, $L \leftarrow 0$, $m \leftarrow -1$, $B(D) \leftarrow 1$, $N \leftarrow 0$.
2. While $(N < n)$ do the following:
 - 2.1 Compute the next discrepancy d . $d \leftarrow (s_N + \sum_{i=1}^L c_i s_{N-i}) \bmod 2$.
 - 2.2 If $d = 1$ then do the following:

$$T(D) \leftarrow C(D), \quad C(D) \leftarrow C(D) + B(D) \cdot D^{N-m}.$$
 If $L \leq N/2$ then $L \leftarrow N+1-L$, $m \leftarrow N$, $B(D) \leftarrow T(D)$.
 - 2.3 $N \leftarrow N+1$.
3. Return(L).

6.31 Note (*intermediate results in Berlekamp-Massey algorithm*) At the end of each iteration of step 2, $\langle L, C(D) \rangle$ is an LFSR of smallest length which generates s^N . Hence, Algorithm 6.30 can also be used to compute the linear complexity profile (Definition 6.23) of a finite sequence.

6.32 Fact The running time of the Berlekamp-Massey algorithm (Algorithm 6.30) for determining the linear complexity of a binary sequence of bitlength n is $O(n^2)$ bit operations.

6.33 Example (*Berlekamp-Massey algorithm*) Table 6.1 shows the steps of Algorithm 6.30 for computing the linear complexity of the binary sequence $s^n = 0, 0, 1, 1, 0, 1, 1, 1, 0$ of length $n = 9$. This sequence is found to have linear complexity 5, and an LFSR which generates it is $\langle 5, 1 + D^3 + D^5 \rangle$. $\square$

6.34 Fact Let s^n be a finite binary sequence of length n , and let the linear complexity of s^n be L . Then there is a unique LFSR of length L which generates s^n if and only if $L \leq \frac{n}{2}$.

An important consequence of Fact 6.34 and Fact 6.24(iii) is the following.

6.35 Fact Let s be an (infinite) binary sequence of linear complexity L , and let t be a (finite) subsequence of s of length at least $2L$. Then the Berlekamp-Massey algorithm (with step 3 modified to return both L and $C(D)$) on input t determines an LFSR of length L which generates s .

s_N	d	$T(D)$	$C(D)$	L	m	$B(D)$	N
—	—	—	1	0	—1	1	0
0	0	—	1	0	—1	1	1
0	0	—	1	0	—1	1	2
1	1	1	$1 + D^3$	3	2	1	3
1	1	$1 + D^3$	$1 + D + D^3$	3	2	1	4
0	1	$1 + D + D^3$	$1 + D + D^2 + D^3$	3	2	1	5
1	1	$1 + D + D^2 + D^3$	$1 + D + D^2$	3	2	1	6
1	0	$1 + D + D^2 + D^3$	$1 + D + D^2$	3	2	1	7
1	1	$1 + D + D^2$	$1 + D + D^2 + D^5$	5	7	$1 + D + D^2$	8
0	1	$1 + D + D^2 + D^5$	$1 + D^3 + D^5$	5	7	$1 + D + D^2$	9

Table 6.1: Steps of the Berlekamp-Massey algorithm of Example 6.33.

6.2.4 Nonlinear feedback shift registers

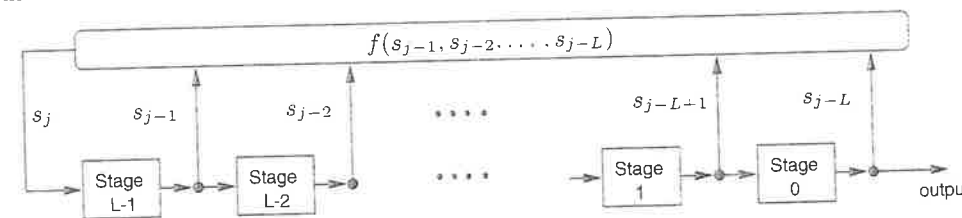
This subsection summarizes selected results about nonlinear feedback shift registers. A function with n binary inputs and one binary output is called a *Boolean function* of n variables; there are 2^{2^n} different Boolean functions of n variables.

6.36 Definition A (general) *feedback shift register (FSR)* of length L consists of L stages (or *delay elements*) numbered $0, 1, \dots, L-1$, each capable of storing one bit and having one input and one output, and a clock which controls the movement of data. During each unit of time the following operations are performed:

- the content of stage 0 is output and forms part of the *output sequence*;
- the content of stage i is moved to stage $i-1$ for each $i, 1 \leq i \leq L-1$; and
- the new content of stage $L-1$ is the *feedback bit* $s_j = f(s_{j-1}, s_{j-2}, \dots, s_{j-L})$, where the *feedback function* f is a Boolean function and s_{j-i} is the previous content of stage $L-i, 1 \leq i \leq L$.

If the initial content of stage i is $s_i \in \{0, 1\}$ for each $0 \leq i \leq L-1$, then $[s_{L-1}, \dots, s_1, s_0]$ is called the *initial state* of the FSR.

Figure 6.7 depicts an FSR. Note that if the feedback function f is a linear function, then the FSR is an LFSR (Definition 6.7). Otherwise, the FSR is called a *nonlinear FSR*.

Figure 6.7: A feedback shift register (FSR) of length L .

6.37 Fact If the initial state of the FSR in Figure 6.7 is $[s_{L-1}, \dots, s_1, s_0]$, then the output sequence $s = s_0, s_1, s_2, \dots$ is uniquely determined by the following recursion:

$$s_j = f(s_{j-1}, s_{j-2}, \dots, s_{j-L}) \text{ for } j \geq L.$$

6.38 Definition An FSR is said to be *non-singular* if and only if every output sequence of the FSR (i.e., for all possible initial states) is periodic.

6.39 Fact An FSR with feedback function $f(s_{j-1}, s_{j-2}, \dots, s_{j-L})$ is non-singular if and only if f is of the form $f = s_{j-L} \oplus g(s_{j-1}, s_{j-2}, \dots, s_{j-L+1})$ for some Boolean function g .

The period of the output sequence of a non-singular FSR of length L is at most 2^L .

6.40 Definition If the period of the output sequence (for any initial state) of a non-singular FSR of length L is 2^L , then the FSR is called a *de Bruijn FSR*, and the output sequence is called a *de Bruijn sequence*.

6.41 Example (de Bruijn sequence) Consider the FSR of length 3 with nonlinear feedback function $f(x_1, x_2, x_3) = 1 \oplus x_2 \oplus x_3 \oplus x_1 x_2$. The following tables show the contents of the 3 stages of the FSR at the end of each unit of time t when the initial state is $[0, 0, 0]$.

t	Stage 2	Stage 1	Stage 0
0	0	0	0
1	1	0	0
2	1	1	0
3	1	1	1

t	Stage 2	Stage 1	Stage 0
4	0	1	1
5	1	0	1
6	0	1	0
7	0	0	1

The output sequence is the de Bruijn sequence with cycle $0, 0, 0, 1, 1, 1, 0, 1$. $\square$

Fact 6.42 demonstrates that the output sequence of de Bruijn FSRs have good statistical properties (compare with Fact 6.14(i)).

6.42 Fact (statistical properties of de Bruijn sequences) Let s be a de Bruijn sequence that is generated by a de Bruijn FSR of length L . Let k be an integer, $1 \leq k \leq L$, and let $\bar{s}$ be any subsequence of s of length $2^L + k - 1$. Then each sequence of length k appears exactly 2^{L-k} times as a subsequence of $\bar{s}$. In other words, the distribution of patterns having fixed length of at most L is uniform.

6.43 Note (converting a maximum-length LFSR to a de Bruijn FSR) Let R_1 be a maximum-length LFSR of length L with (linear) feedback function $f(s_{j-1}, s_{j-2}, \dots, s_{j-L})$. Then the FSR R_2 with feedback function $g(s_{j-1}, s_{j-2}, \dots, s_{j-L}) = f \oplus \bar{s}_{j-1} \bar{s}_{j-2} \dots \bar{s}_{j-L+1}$ is a de Bruijn FSR. Here, $\bar{s}_i$ denotes the complement of s_i . The output sequence of R_2 is obtained from that of R_1 by simply adding a 0 to the end of each subsequence of $L-1$ 0's occurring in the output sequence of R_1 .

6.3 Stream ciphers based on LFSRs

As mentioned in the beginning of §6.2.1, linear feedback shift registers are widely used in keystream generators because they are well-suited for hardware implementation, produce sequences having large periods and good statistical properties, and are readily analyzed using algebraic techniques. Unfortunately, the output sequences of LFSRs are also easily predictable, as the following argument shows. Suppose that the output sequence s of an LFSR has linear complexity L . The connection polynomial $C(D)$ of an LFSR of length L which generates s can be efficiently determined using the Berlekamp-Massey algorithm

(Algorithm 6.30) from any (short) subsequence t of s having length at least $n = 2L$ (cf. Fact 6.35). Having determined $C(D)$, the LFSR $\langle L, C(D) \rangle$ can then be initialized with any substring of t having length L , and used to generate the remainder of the sequence s . An adversary may obtain the required subsequence t of s by mounting a known or chosen-plaintext attack (§1.13.1) on the stream cipher: if the adversary knows the plaintext subsequence $m_1, m_2, \dots, m_n$ corresponding to a ciphertext sequence $c_1, c_2, \dots, c_n$, the corresponding keystream bits are obtained as $m_i \oplus c_i$, $1 \leq i \leq n$.

6.44 Note (*use of LFSRs in keystream generators*) Since a well-designed system should be secure against known-plaintext attacks, an LFSR should never be used by itself as a keystream generator. Nevertheless, LFSRs are desirable because of their very low implementation costs. Three general methodologies for destroying the linearity properties of LFSRs are discussed in this section:

- (i) using a nonlinear combining function on the outputs of several LFSRs (§6.3.1);
- (ii) using a nonlinear filtering function on the contents of a single LFSR (§6.3.2); and
- (iii) using the output of one (or more) LFSRs to control the clock of one (or more) other LFSRs (§6.3.3).

Desirable properties of LFSR-based keystream generators

For essentially all possible secret keys, the output sequence of an LFSR-based keystream generator should have the following properties:

- 1. large period;
- 2. large linear complexity; and
- 3. good statistical properties (e.g., as described in Fact 6.14).

It is emphasized that these properties are only *necessary* conditions for a keystream generator to be considered cryptographically secure. Since mathematical proofs of security of such generators are not known, such generators can only be deemed *computationally secure* (§1.13.3(iv)) after having withstood sufficient public scrutiny.

6.45 Note (*connection polynomial*) Since a desirable property of a keystream generator is that its output sequences have large periods, component LFSRs should always be chosen to be maximum-length LFSRs, i.e., the LFSRs should be of the form $\langle L, C(D) \rangle$ where $C(D) \in \mathbb{Z}_2[D]$ is a primitive polynomial of degree L (see Definition 6.13 and Fact 6.12(ii)).

6.46 Note (*known vs. secret connection polynomial*) The LFSRs in an LFSR-based keystream generator may have *known* or *secret* connection polynomials. For known connections, the secret key generally consists of the initial contents of the component LFSRs. For secret connections, the secret key for the keystream generator generally consists of both the initial contents and the connections.

For LFSRs of length L with secret connections, the connection polynomials should be selected uniformly at random from the set of all primitive polynomials of degree L over $\mathbb{Z}_2$. Secret connections are generally recommended over known connections as the former are more resistant to certain attacks which use precomputation for analyzing the particular connection, and because the former are more amenable to statistical analysis. Secret connection LFSRs have the drawback of requiring extra circuitry to implement in hardware. However, because of the extra security possible with secret connections, this cost may sometimes be compensated for by choosing shorter LFSRs.

6.47 Note (*sparse vs. dense connection polynomial*) For implementation purposes, it is advantageous to choose an LFSR that is *sparse*; i.e., only a few of the coefficients of the connection polynomial are non-zero. Then only a small number of connections must be made between the stages of the LFSR in order to compute the feedback bit. For example, the connection polynomial might be chosen to be a primitive trinomial (cf. Table 4.8). However, in some LFSR-based keystream generators, special attacks can be mounted if sparse connection polynomials are used. Hence, it is generally recommended not to use sparse connection polynomials in LFSR-based keystream generators.

6.3.1 Nonlinear combination generators

One general technique for destroying the linearity inherent in LFSRs is to use several LFSRs in parallel. The keystream is generated as a nonlinear function f of the outputs of the component LFSRs; this construction is illustrated in Figure 6.8. Such keystream generators are called *nonlinear combination generators*, and f is called the *combining function*. The remainder of this subsection demonstrates that the function f must satisfy several criteria in order to withstand certain particular cryptographic attacks.

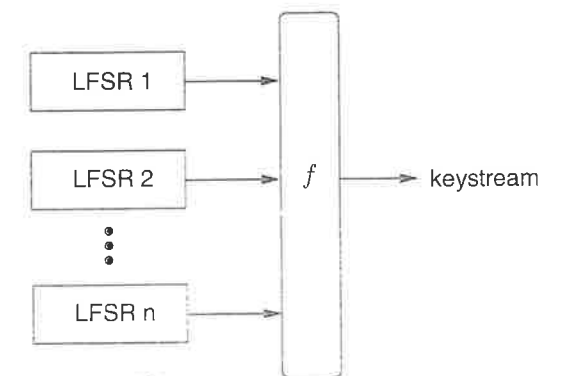


Figure 6.8: A nonlinear combination generator. f is a nonlinear combining function.

6.48 Definition A product of m distinct variables is called an m^{th} order product of the variables. Every Boolean function $f(x_1, x_2, \dots, x_n)$ can be written as a modulo 2 sum of distinct m^{th} order products of its variables, $0 \leq m \leq n$; this expression is called the *algebraic normal form* of f . The *nonlinear order* of f is the maximum of the order of the terms appearing in its algebraic normal form.

For example, the Boolean function $f(x_1, x_2, x_3, x_4, x_5) = 1 \oplus x_2 \oplus x_3 \oplus x_4 x_5 \oplus x_1 x_3 x_4 x_5$ has nonlinear order 4. Note that the maximum possible nonlinear order of a Boolean function in n variables is n . Fact 6.49 demonstrates that the output sequence of a nonlinear combination generator has high linear complexity, provided that a combining function f of high nonlinear order is employed.

6.49 Fact Suppose that n maximum-length LFSRs, whose lengths $L_1, L_2, \dots, L_n$ are pairwise distinct and greater than 2, are combined by a nonlinear function $f(x_1, x_2, \dots, x_n)$ (as in Figure 6.8) which is expressed in algebraic normal form. Then the linear complexity of the keystream is $f(L_1, L_2, \dots, L_n)$. (The expression $f(L_1, L_2, \dots, L_n)$ is evaluated over the integers rather than over $\mathbb{Z}_2$.)

sists of the initial states of the LFSRs, and an initial (integer) carry C_0 . The keystream is generated as follows. At time j ($j \geq 1$), the LFSRs are stepped producing output bits $x_1, x_2, \dots, x_n$, and the integer sum $S_j = \sum_{i=1}^n x_i + C_{j-1}$ is computed. The keystream bit is $S_j \bmod 2$ (the least significant bit of S_j), while the new carry is computed as $C_j = \lfloor S_j/2 \rfloor$ (the remaining bits of S_j). The period of the keystream is $\prod_{i=1}^n (2^{L_i} - 1)$, while its linear complexity is close to this number.

Even though the summation generator has high period, linear complexity, and correlation immunity, it is vulnerable to certain correlation attacks and a known-plaintext attack based on its 2-adic span (see page 218). $\square$

6.3.2 Nonlinear filter generators

Another general technique for destroying the linearity inherent in LFSRs is to generate the keystream as some nonlinear function of the stages of a single LFSR; this construction is illustrated in Figure 6.11. Such keystream generators are called *nonlinear filter generators*, and f is called the *filtering function*.

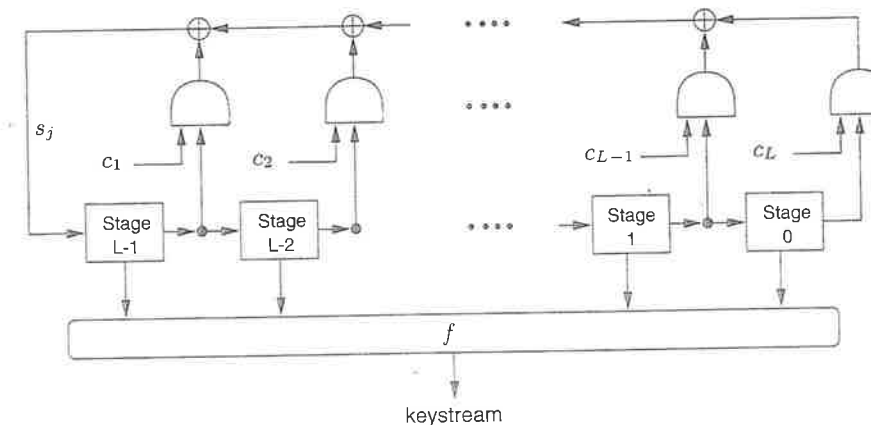


Figure 6.11: A nonlinear filter generator. f is a nonlinear Boolean filtering function.

Fact 6.55 describes the linear complexity of the output sequence of a nonlinear filter generator.

6.55 Fact Suppose that a nonlinear filter generator is constructed using a maximum-length LFSR of length L and a filtering function f of nonlinear order m (as in Figure 6.11).

- (i) (*Key's bound*) The linear complexity of the keystream is at most $L_m = \sum_{i=1}^m \binom{L}{i}$.
- (ii) For a fixed maximum-length LFSR of prime length L , the fraction of Boolean functions f of nonlinear order m which produce sequences of maximum linear complexity L_m is

$$P_m \approx \exp(-L_m/(L \cdot 2^L)) > e^{-1/L}.$$

Therefore, for large L , most of the generators produce sequences whose linear complexity meets the upper bound in (i).

The nonlinear function f selected for a filter generator should include many terms of each order up to the nonlinear order of f .

6.56 Example (knapsack generator) The knapsack keystream generator is defined by a maximum-length LFSR $\langle L, C(D) \rangle$ and a modulus $Q = 2^L$. The secret key consists of L knapsack integer weights $a_1, a_2, \dots, a_L$ each of bitlength L , and the initial state of the LFSR. Recall that the subset sum problem (§3.10) is to determine a subset of the knapsack weights which add up to a given integer s , provided that such a subset exists; this problem is NP-hard (Fact 3.91). The keystream is generated as follows: at time j , the LFSR is stepped and the knapsack sum $S_j = \sum_{i=1}^L x_i a_i \bmod Q$ is computed, where $[x_L, \dots, x_2, x_1]$ is the state of the LFSR at time j . Finally, selected bits of S_j (after S_j is converted to its binary representation) are extracted to form part of the keystream (the $\lceil \lg L \rceil$ least significant bits of S_j should be discarded). The linear complexity of the keystream is then virtually certain to be $L(2^L - 1)$.

Since the state of an LFSR is a binary vector, the function which maps the LFSR state to the knapsack sum S_j is indeed nonlinear. Explicitly, let the function f be defined by $f(x) = \sum_{i=1}^L x_i a_i \bmod Q$, where $x = [x_L, \dots, x_2, x_1]$ is a state. If x and y are two states then, in general, $f(x \oplus y) \neq f(x) + f(y)$. $\square$

6.3.3 Clock-controlled generators

In nonlinear combination generators and nonlinear filter generators, the component LFSRs are clocked regularly; i.e., the movement of data in all the LFSRs is controlled by the same clock. The main idea behind a *clock-controlled generator* is to introduce nonlinearity into LFSR-based keystream generators by having the output of one LFSR control the *clocking* (i.e., stepping) of a second LFSR. Since the second LFSR is clocked in an irregular manner the hope is that attacks based on the regular motion of LFSRs can be foiled. Two clock-controlled generators are described in this subsection: (i) the alternating step generator and (ii) the shrinking generator.

(i) The alternating step generator

The alternating step generator uses an LFSR R_1 to control the stepping of two LFSRs, R_2 and R_3 . The keystream produced is the XOR of the output sequences of R_2 and R_3 .

6.57 Algorithm Alternating step generator

SUMMARY: a control LFSR R_1 is used to selectively step two other LFSRs, R_2 and R_3 .
OUTPUT: a sequence which is the bitwise XOR of the output sequences of R_2 and R_3 .

The following steps are repeated until a keystream of desired length is produced.

1. Register R_1 is clocked.
2. If the output of R_1 is 1 then:
 R_2 is clocked; R_3 is not clocked but its previous output bit is repeated.
 (For the first clock cycle, the "previous output bit" of R_3 is taken to be 0.)
3. If the output of R_1 is 0 then:
 R_3 is clocked; R_2 is not clocked but its previous output bit is repeated.
 (For the first clock cycle, the "previous output bit" of R_2 is taken to be 0.)
4. The output bits of R_2 and R_3 are XORed; the resulting bit is part of the keystream.

More formally, let the output sequences of LFSRs R_1 , R_2 , and R_3 be $a_0, a_1, a_2, \dots$, $b_0, b_1, b_2, \dots$, and $c_0, c_1, c_2, \dots$, respectively. Define $b_{-1} = c_{-1} = 0$. Then the keystream produced by the alternating step generator is $x_0, x_1, x_2, \dots$, where $x_j = b_{t(j)} \oplus c_{j-t(j)-1}$.

and $t(j) = (\sum_{i=0}^j a_i) - 1$ for all $j \geq 0$. The alternating step generator is depicted in Figure 6.12.

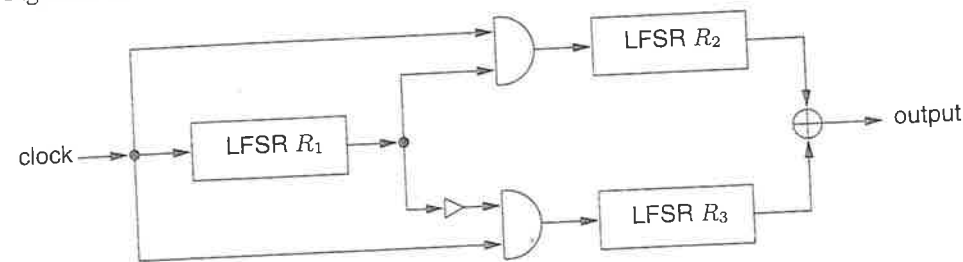


Figure 6.12: The alternating step generator.

6.58 Example (alternating step generator with artificially small parameters) Consider an alternating step generator with component LFSRs $R_1 = \langle 3, 1 + D^2 + D^3 \rangle$, $R_2 = \langle 4, 1 + D^3 + D^4 \rangle$, and $R_3 = \langle 5, 1 + D + D^3 + D^4 + D^5 \rangle$. Suppose that the initial states of R_1 , R_2 , and R_3 are $[0, 0, 1]$, $[1, 0, 1, 1]$, and $[0, 1, 0, 0, 1]$, respectively. The output sequence of R_1 is the 7-periodic sequence with cycle

$$a^7 = 1, 0, 0, 1, 0, 1, 1.$$

The output sequence of R_2 is the 15-periodic sequence with cycle

$$b^{15} = 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0.$$

The output sequence of R_3 is the 31-periodic sequence with cycle

$$c^{31} = 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0.$$

The keystream generated is

$$x = 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, \dots \quad \square$$

Fact 6.59 establishes, under the assumption that R_1 produces a de Bruijn sequence (see Definition 6.40), that the output sequence of an alternating step generator satisfies the basic requirements of high period, high linear complexity, and good statistical properties.

6.59 Fact (properties of the alternating step generator) Suppose that R_1 produces a de Bruijn sequence of period 2^{L_1} . Furthermore, suppose that R_2 and R_3 are maximum-length LFSRs of lengths L_2 and L_3 , respectively, such that $\gcd(L_2, L_3) = 1$. Let x be the output sequence of the alternating step generator formed by R_1 , R_2 , and R_3 .

(i) The sequence x has period $2^{L_1} \cdot (2^{L_2} - 1) \cdot (2^{L_3} - 1)$.

(ii) The linear complexity $L(x)$ of x satisfies

$$(L_2 + L_3) \cdot 2^{L_1 - 1} < L(x) \leq (L_2 + L_3) \cdot 2^{L_1}.$$

(iii) The distribution of patterns in x is almost uniform. More precisely, let P be any binary string of length t bits, where $t \leq \min(L_2, L_3)$. If $x(t)$ denotes any t consecutive bits in x , then the probability that $x(t) = P$ is $(\frac{1}{2})^t + O(1/2^{L_2-t}) + O(1/2^{L_3-t})$.

Since a de Bruijn sequence can be obtained from the output sequence s of a maximum-length LFSR (of length L) by simply adding a 0 to the end of each subsequence of $L-1$ 0's occurring in s (see Note 6.43), it is reasonable to expect that the assertions of high period,

high linear complexity, and good statistical properties in Fact 6.59 also hold when R_1 is a maximum-length LFSR. Note, however, that this has not yet been proven.

6.60 Note (security of the alternating step generator) The LFSRs R_1 , R_2 , R_3 should be chosen to be maximum-length LFSRs whose lengths L_1 , L_2 , L_3 are pairwise relatively prime: $\gcd(L_1, L_2) = 1$, $\gcd(L_2, L_3) = 1$, $\gcd(L_1, L_3) = 1$. Moreover, the lengths should be about the same. If $L_1 \approx l$, $L_2 \approx l$, and $L_3 \approx l$, the best known attack on the alternating step generator is a divide-and-conquer attack on the control register R_1 which takes approximately 2^l steps. Thus, if $l \approx 128$, the generator is secure against all presently known attacks.

(ii) The shrinking generator

The shrinking generator is a relatively new keystream generator, having been proposed in 1993. Nevertheless, due to its simplicity and provable properties, it is a promising candidate for high-speed encryption applications. In the shrinking generator, a control LFSR R_1 is used to select a portion of the output sequence of a second LFSR R_2 . The keystream produced is, therefore, a *shrunk* version (also known as an *irregularly decimated subsequence*) of the output sequence of R_2 , as specified in Algorithm 6.61 and depicted in Figure 6.13.

6.61 Algorithm Shrinking generator

SUMMARY: a control LFSR R_1 is used to control the output of a second LFSR R_2 . The following steps are repeated until a keystream of desired length is produced.

1. Registers R_1 and R_2 are clocked.
2. If the output of R_1 is 1, the output bit of R_2 forms part of the keystream.
3. If the output of R_1 is 0, the output bit of R_2 is discarded.

More formally, let the output sequences of LFSRs R_1 and R_2 be $a_0, a_1, a_2, \dots$ and $b_0, b_1, b_2, \dots$, respectively. Then the keystream produced by the shrinking generator is $x_0, x_1, x_2, \dots$, where $x_j = b_{i_j}$, and, for each $j \geq 0$, i_j is the position of the j^{th} 1 in the sequence $a_0, a_1, a_2, \dots$.

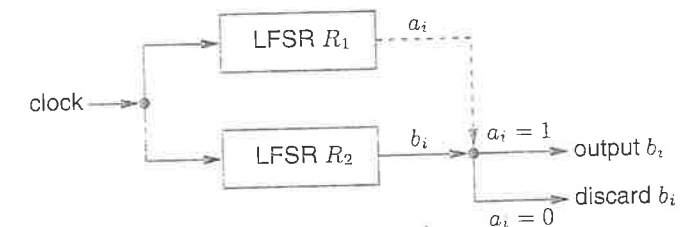


Figure 6.13: The shrinking generator.

6.62 Example (shrinking generator with artificially small parameters) Consider a shrinking generator with component LFSRs $R_1 = \langle 3, 1 + D + D^3 \rangle$ and $R_2 = \langle 5, 1 + D^3 + D^5 \rangle$. Suppose that the initial states of R_1 and R_2 are $[1, 0, 0]$ and $[0, 0, 1, 0, 1]$, respectively. The output sequence of R_1 is the 7-periodic sequence with cycle

$$a^7 = 0, 0, 1, 1, 1, 0, 1,$$

while the output sequence of R_2 is the 31-periodic sequence with cycle

$$b^{31} = 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0.$$

The keystream generated is

$$x = 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, \dots \quad \square$$

Fact 6.63 establishes that the output sequence of a shrinking generator satisfies the basic requirements of high period, high linear complexity, and good statistical properties.

6.63 Fact (*properties of the shrinking generator*) Let R_1 and R_2 be maximum-length LFSRs of lengths L_1 and L_2 , respectively, and let x be an output sequence of the shrinking generator formed by R_1 and R_2 .

- (i) If $\gcd(L_1, L_2) = 1$, then x has period $(2^{L_2} - 1) \cdot 2^{L_1-1}$.
- (ii) The linear complexity $L(x)$ of x satisfies

$$L_2 \cdot 2^{L_1-2} < L(x) \leq L_2 \cdot 2^{L_1-1}.$$

- (iii) Suppose that the connection polynomials for R_1 and R_2 are chosen uniformly at random from the set of all primitive polynomials of degrees L_1 and L_2 over $\mathbb{Z}_2$. Then the distribution of patterns in x is almost uniform. More precisely, if P is any binary string of length t bits and $x(t)$ denotes any t consecutive bits in x , then the probability that $x(t) = P$ is $(\frac{1}{2})^t + O(t/2^{L_2})$.

6.64 Note (*security of the shrinking generator*) Suppose that the component LFSRs R_1 and R_2 of the shrinking generator have lengths L_1 and L_2 , respectively. If the connection polynomials for R_1 and R_2 are known (but not the initial contents of R_1 and R_2), the best attack known for recovering the secret key takes $O(2^{L_1} \cdot L_2^3)$ steps. On the other hand, if secret (and variable) connection polynomials are used, the best attack known takes $O(2^{2L_1} \cdot L_1 \cdot L_2)$ steps. There is also an attack through the linear complexity of the shrinking generator which takes $O(2^{L_1} \cdot L_2^2)$ steps (regardless of whether the connections are known or secret), but this attack requires $2^{L_1} \cdot L_2$ consecutive bits from the output sequence and is, therefore, infeasible for moderately large L_1 and L_2 . For maximum security, R_1 and R_2 should be maximum-length LFSRs, and their lengths should satisfy $\gcd(L_1, L_2) = 1$. Moreover, secret connections should be used. Subject to these constraints, if $L_1 \approx l$ and $L_2 \approx l$, the shrinking generator has a security level approximately equal to 2^{2l} . Thus, if $L_1 \approx 64$ and $L_2 \approx 64$, the generator appears to be secure against all presently known attacks.

6.4 Other stream ciphers

While the LFSR-based stream ciphers discussed in §6.3 are well-suited to hardware implementation, they are not especially amenable to software implementation. This has led to several recent proposals for stream ciphers designed particularly for fast software implementation. Most of these proposals are either proprietary, or are relatively new and have not received sufficient scrutiny from the cryptographic community; for this reason, they are not presented in this section, and instead only mentioned in the chapter notes on page 222.

Two promising stream ciphers specifically designed for fast software implementation are SEAL and RC4. SEAL is presented in §6.4.1. RC4 is used in commercial products, and has a variable key-size, but it remains proprietary and is not presented here. Two

other widely used stream ciphers not based on LFSRs are the Output Feedback (OFB; see §7.2.2(iv)) and Cipher Feedback (CFB; see §7.2.2(iii)) modes of block ciphers. Another class of keystream generators not based on LFSRs are those whose security relies on the intractability of an underlying number-theoretic problem; these generators are much slower than those based on LFSRs and are discussed in §5.5.

6.4.1 SEAL

SEAL (Software-optimized Encryption Algorithm) is a binary additive stream cipher (see Definition 6.4) that was proposed in 1993. Since it is relatively new, it has not yet received much scrutiny from the cryptographic community. However, it is presented here because it is one of the few stream ciphers that was specifically designed for efficient software implementation and, in particular, for 32-bit processors.

SEAL is a length-increasing pseudorandom function which maps a 32-bit *sequence number* n to an L -bit keystream under control of a 160-bit secret key a . In the preprocessing stage (step 1 of Algorithm 6.68), the key is stretched into larger tables using the table-generation function G_a specified in Algorithm 6.67; this function is based on the Secure Hash Algorithm SHA-1 (Algorithm 9.53). Subsequent to this preprocessing, keystream generation requires about 5 machine instructions per byte, and is an order of magnitude faster than DES (Algorithm 7.82).

The following notation is used in SEAL for 32-bit quantities A, B, C, D, X_i , and Y_j :

- $\bar{A}$: bitwise complement of A
- $A \wedge B, A \vee B, A \oplus B$: bitwise AND, inclusive-OR, exclusive-OR
- " $A \leftarrow s$ ": 32-bit result of rotating A left through s positions
- " $A \rightarrow s$ ": 32-bit result of rotating A right through s positions
- $A + B$: mod 2^{32} sum of the unsigned integers A and B
- $f(B, C, D) \stackrel{\text{def}}{=} (B \wedge C) \vee (\bar{B} \wedge D)$; $g(B, C, D) \stackrel{\text{def}}{=} (B \wedge C) \vee (B \wedge D) \vee (C \wedge D)$;
 $h(B, C, D) \stackrel{\text{def}}{=} B \oplus C \oplus D$
- $A \| B$: concatenation of A and B
- $(X_1, \dots, X_j) \leftarrow (Y_1, \dots, Y_j)$: simultaneous assignments $(X_i \leftarrow Y_i)$, where $(Y_1, \dots, Y_j)$ is evaluated prior to any assignments.

6.65 Note (*SEAL 1.0 vs. SEAL 2.0*) The table-generation function (Algorithm 6.67) for the first version of SEAL (SEAL 1.0) was based on the Secure Hash Algorithm (SHA). SEAL 2.0 differs from SEAL 1.0 in that the table-generation function for the former is based on the modified Secure Hash Algorithm SHA-1 (Algorithm 9.53).

6.66 Note (*tables*) The table generation (step 1 of Algorithm 6.68) uses the compression function of SHA-1 to expand the secret key a into larger tables T, S , and R . These tables can be precomputed, but only after the secret key a has been established. Tables T and S are 2K bytes and 1K byte in size, respectively. The size of table R depends on the desired bitlength L of the keystream — each 1K byte of keystream requires 16 bytes of R .